Subject: ACS 547, Noise Control Applications - Module 2 Assignment

Date: March 29, 2025 (Submitted)

Problem 1 - Washing Machine Mount Design (1 DOF)

Problem 1a

The Matlab code for this problem is listed in Appendix 1.

Figure 1 shows the load force produced by the 1 kg imbalanced mass at a 0.5 m radius from the center of the washing machine as a function of angular speed in RPM.

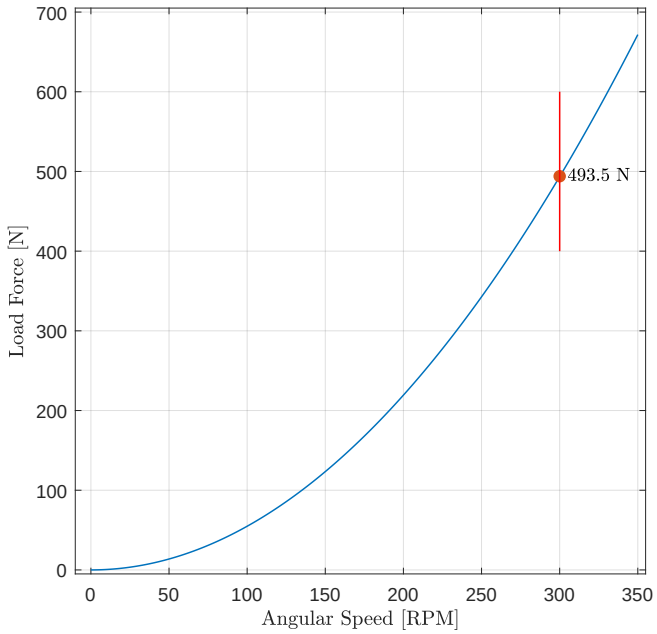

Figure 1: Load force of 1 kg imbalanced mass at a 0.5 m radius versus washing machine angular speed.

The force amplitude is 493.5 N at a frequency of 5 Hz when the washing machine operates at 300 RPM.

Problem 1b

The static displacement of the load in the washing machine must be less than 1 cm (i.e., the deflection is calculated with only the 10 kg load mass). The corresponding mount stiffness, k_s, is **9,810** $\frac{N}{m}$.

The natural frequency, w_o, of the mount is **1.5 Hz** (or 9.4 $\frac{\text{radians}}{s}$).

Problem 1c

Figure 2 shows the transmission loss of the system for a set of damping ratios ranging from 0.001 to 1.

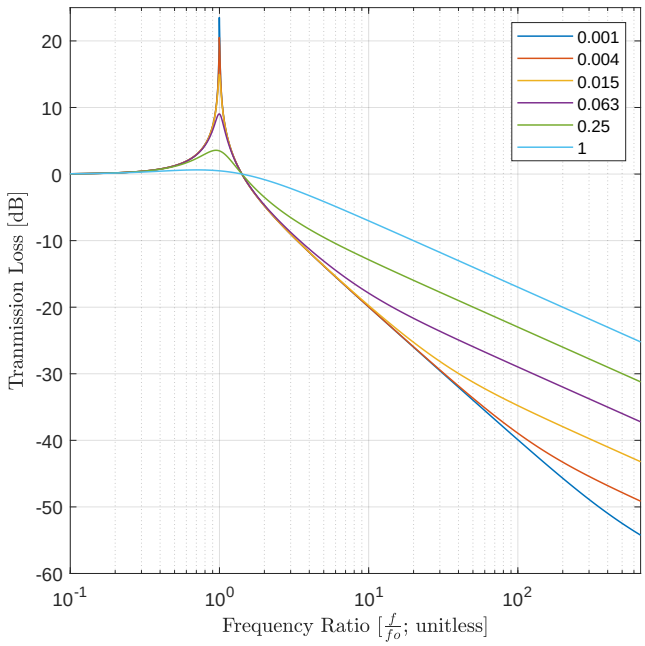

Figure 2: Transmission loss of the system. Its natural frequency is 1.5 Hz.

Problem 1d

Figure 3 shows the force applied to the foundation as a function of RPM.

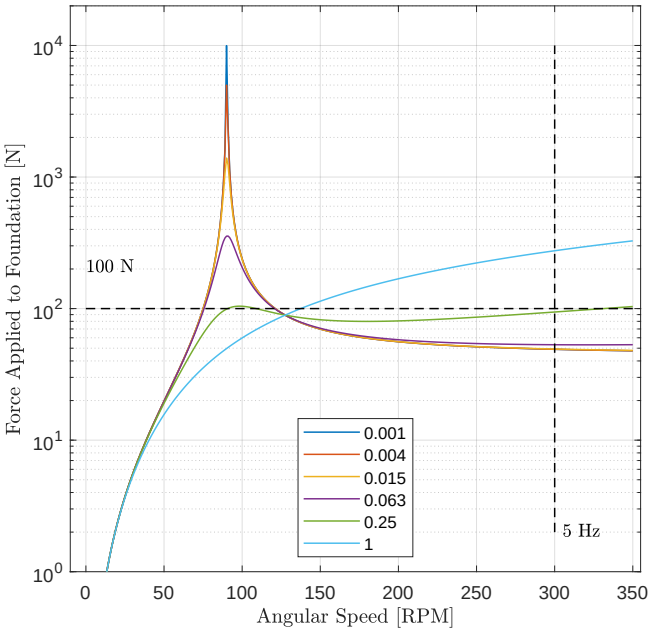

Figure 3: Force applied to the foundation versus RPM.

There is no damping ratio for which the applied force is less than 100 N across the whole operating range. The best damping ratio is $\zeta = 0.25$. With this damping ratio, the force applied to the foundation still

exceeds the 100 N target in two regions: 91 to 109 RPM and 331 to 350 RPM. In both regions the force is about 104 N.

Problem 1e

Figure 4 shows the admittance as a function of RPM.

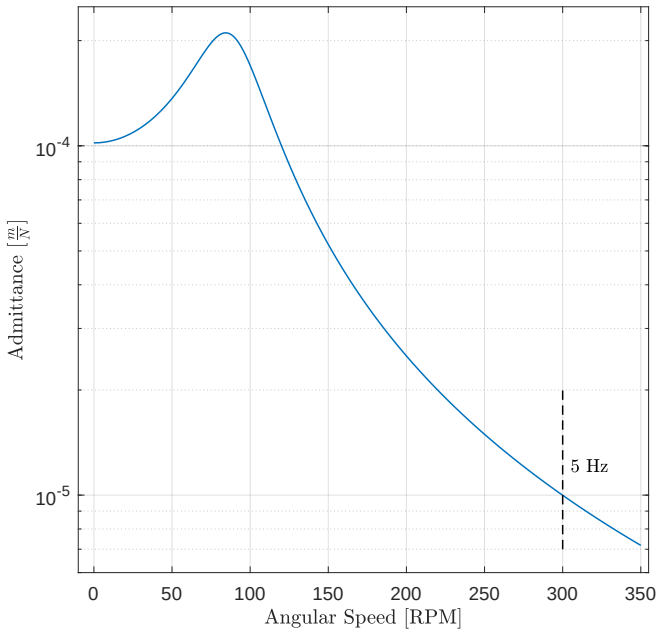

Figure 4: Admittance versus RPM.

The admittance at 300 RPM is 1e-5 $\frac{\mathrm{m}}{\mathrm{N}}$.

Problem 1f

Figure 5 shows the displacement of the washing machine as a function of RPM.
The displacement of the washing machine at 300 RPM is 5×10^{-3} m. This amount of displacement for the 300 RPM operating state seems excessive. The washing machine undergoes larger momentary displacements as is spins up to the 300 RPM operating state, in particular around 100 RPM.

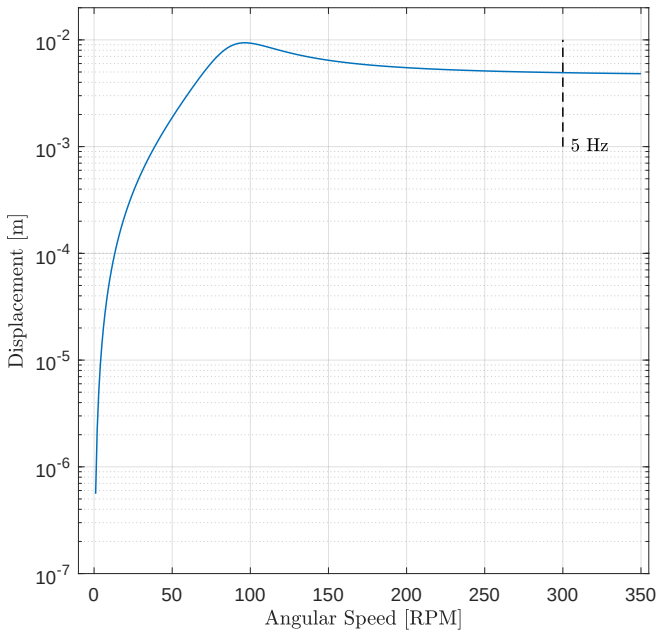

Figure 5: Displacement versus RPM.

Problem 2 - Washing Machine Mount Design (2 DOF)

Problem 2a

The Matlab code for this problem is listed in Appendix 2.

Figure 1 illustrates the two-stage vibration mount considered here (see slide 8 of 42 for Lecture 14 on Monday, March 3, 2025). Modeling will be done using discrete viscous damping elements.

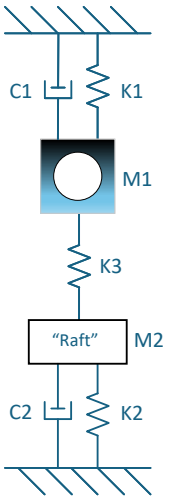

Figure 1: Two-stage vibration mount system for the washing machine.

Problem 2b

Table 1 lists the initial stiffness and damping coefficients for the system.

M1	110	kg
K1	100	$\frac{\text{N}}{\text{m}}$
C1	5	$\frac{\text{kg}}{\text{s}}$

M2	100	kg
K2	100	$\frac{\text{N}}{\text{m}}$
C2	5	$\frac{\text{kg}}{\text{s}}$

M3	0	kg
K3	1,000	$\frac{\text{N}}{\text{m}}$
C3	5	$\frac{\text{kg}}{\text{s}}$

Table 1: Initial values of stiffness and damping coefficients.

Problem 2c

Figure 2 illustrates the admittance of the washing machine and the raft with the force applied to the washing machine.

At the operating speed of 300 RPM, the admittance of the washing machine is 9.31×10^{-6} $\frac{\text{m}}{\text{N}}$.

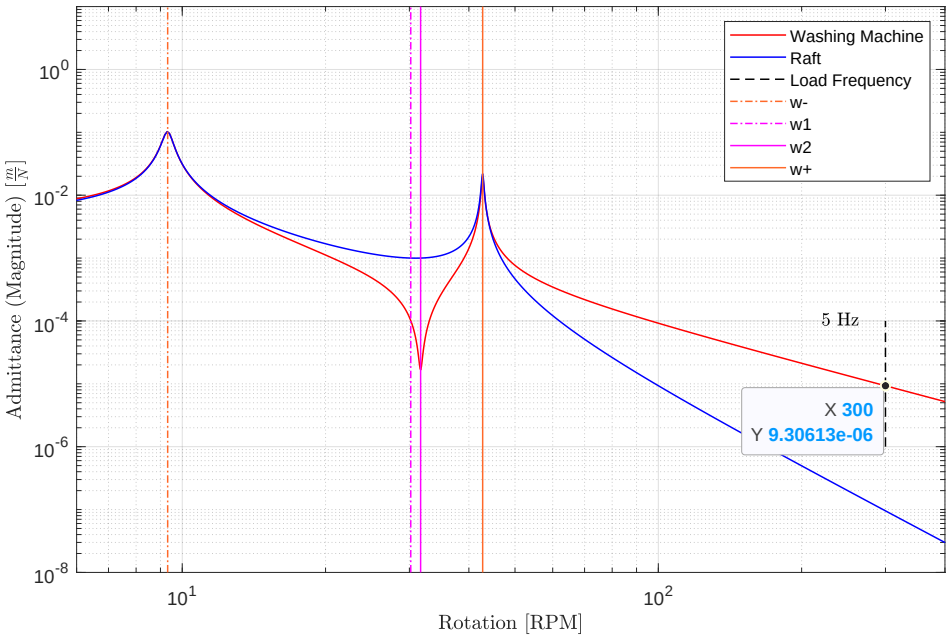

Figure 2: Admittance of the washing machine and the raft with a force on the washing machine.

Problem 2d

Figure 3 illustrates the displacement of the washing machine and the raft with the force applied to the washing machine.

At the operating speed of 300 RPM, the displacement of the washing machine is 4.6×10^{-3} m, which is less than the 5.0×10^{-3} m displacement with the first-order system.

The displacement of the raft is 4.7×10^{-5} m.

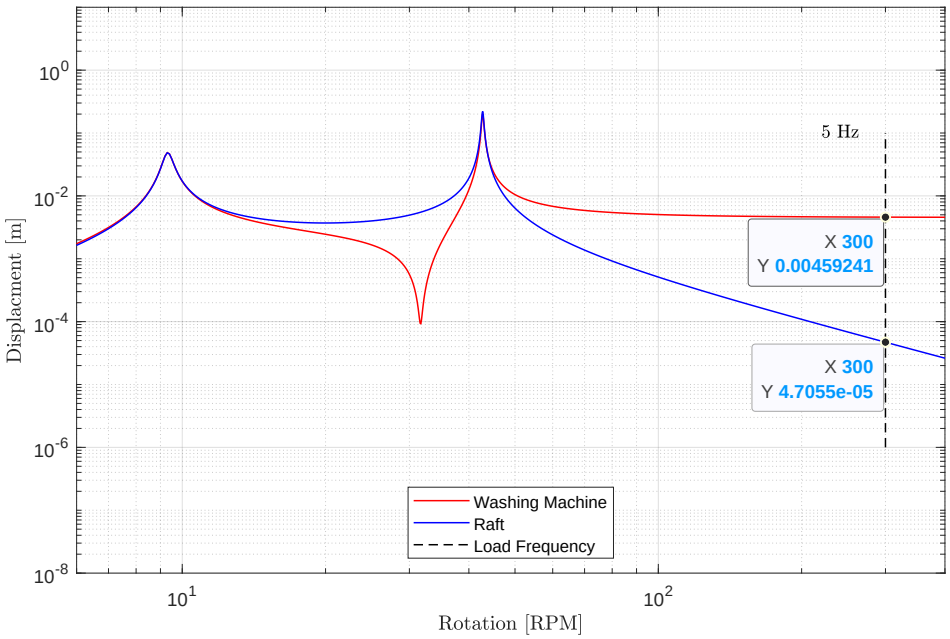

Figure 3: Displacement of the washing machine and the raft with a force on the washing machine.

Problem 2e

Figure 4 illustrates the force applied to the ground by the raft. This is calculated by multiplying the displacement of the raft by the K2 spring constant.

The load on the ground is always less than 100 N.

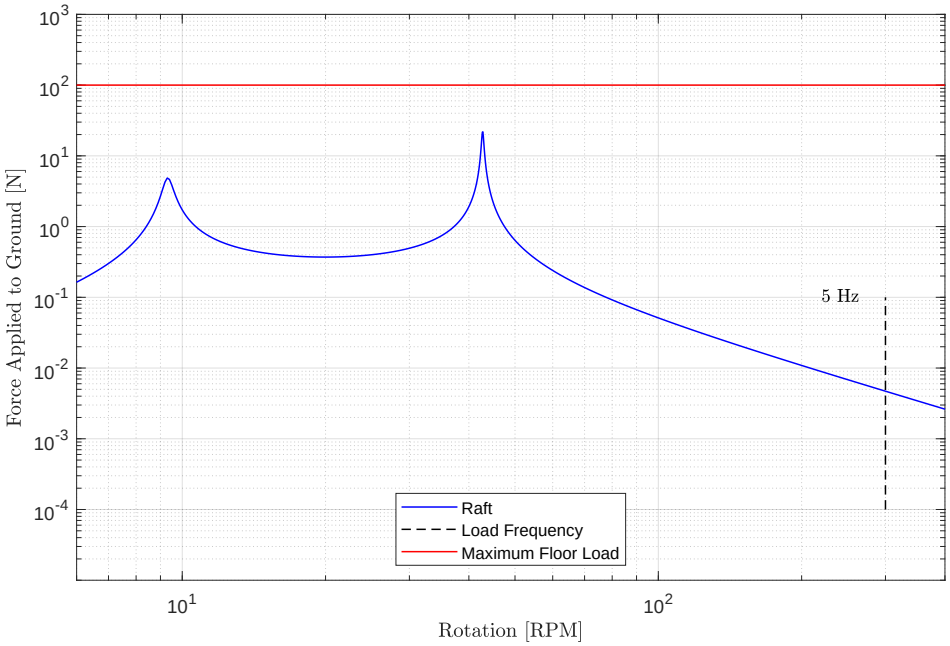

Figure 4: Force applied to the ground by the raft.

Problem 2f

The initial parameter set chosen, see Table **??**, facilitated a displacement of the washing machine of 4.6×10^{-3} m at 300 RPM, which is less than the 5.0×10^{-3} m found for the 1 DOF system. The force applied to the ground by the raft is less than the required 100 N maximum.

Problem 2g

The initial parameter set chosen, see Table **??**, facilitated a displacement of the washing machine of 4.6×10^{-3} m at 300 RPM, which is less than the 5.0×10^{-3} m found for the 1 DOF system. The force applied to the ground by the raft is less than the required 100 N maximum.

Problem 3 - Washing Machine Dynamic Vibration Absorber Design

The Matlab code for this problem is listed in Appendix 3.

Part 3a

Figure 1 illustrates the dynamic vibration absorber (DVA) system.

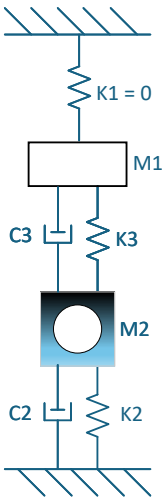

Figure 1: Dynamic vibration absorber system for the washing machine.

Table 2 lists the stiffness and damping coefficients for the system.

M1	15	kg
K1	0	$\frac{N}{m}$
C1	0	$\frac{kg}{s}$

M2	110	kg
K2	9,810	$\frac{N}{m}$
C2	1,962	$\frac{kg}{s}$ [1]

M3	0	kg
K3	1e5	$\frac{N}{m}$
C3	1e4	$\frac{kg}{s}$ [2]

Table 2: Stiffness and damping coefficients for the DVA system.

Damping was implemented by adding with an imaginary component to the respective spring stiffness[3].

[1] 0.2 fraction of respective spring stiffness.
[2] 0.1 fraction of respective spring stiffness.
[3] Not equivalent to using a damping vector in the argument to the given Matlab nDOF function script-file.

Part 3b

Figure 2 shows the admittance response of the original system and DVA system based on the equations presented in the tutorial lecture on DVA system analysis.

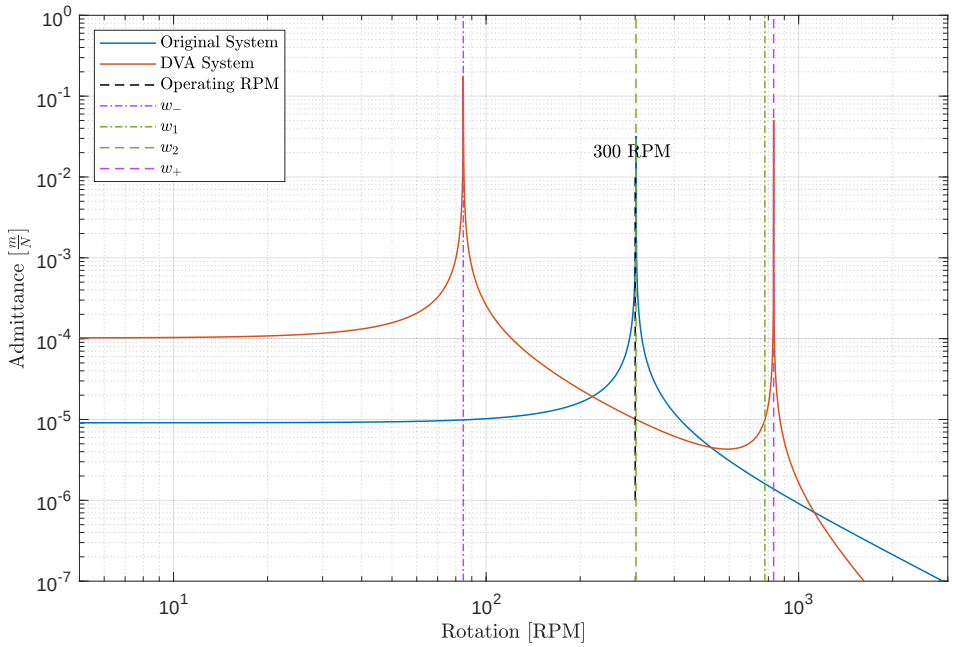

Figure 2: Admittance responses based on the equations from the DVA tutorial lecture.

Part 3c

Figure 3 shows the admittance response of the DVA system using the nDOF function script-file. The stiffness of the coupling spring, k_3, is set to place the null of the admittance response at the 300 RPM operating state of the washing machine.

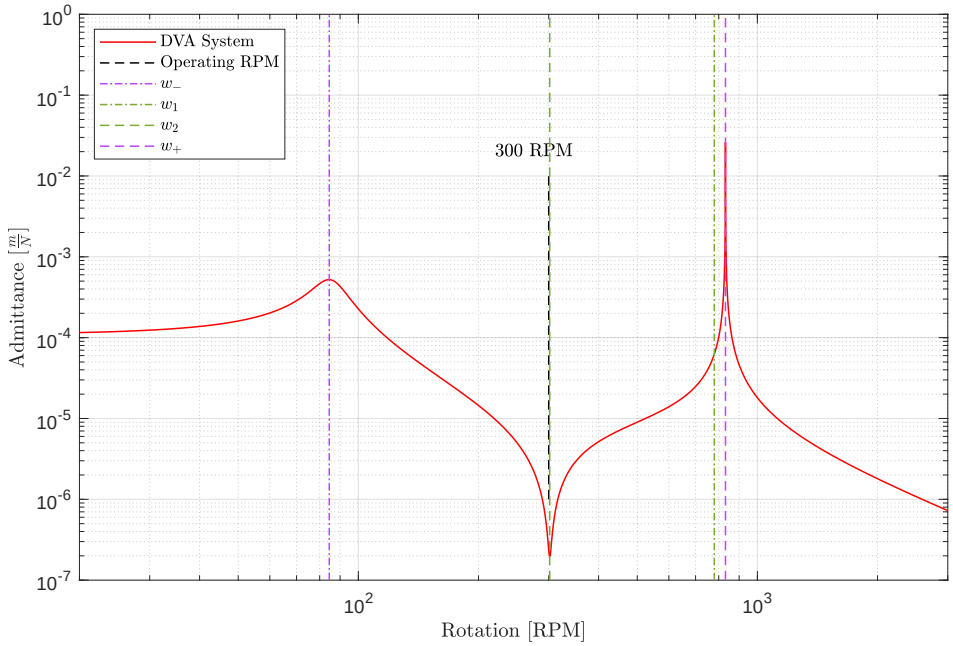

Figure 3: Solution using nDOF script-file function.

Added damping to the DVA system reduces the prominence of its two resonant peaks.

Part 3d

Figure 4 ph

The ramp up to 300 RPM occurs quickly so that the large displacements are transient.

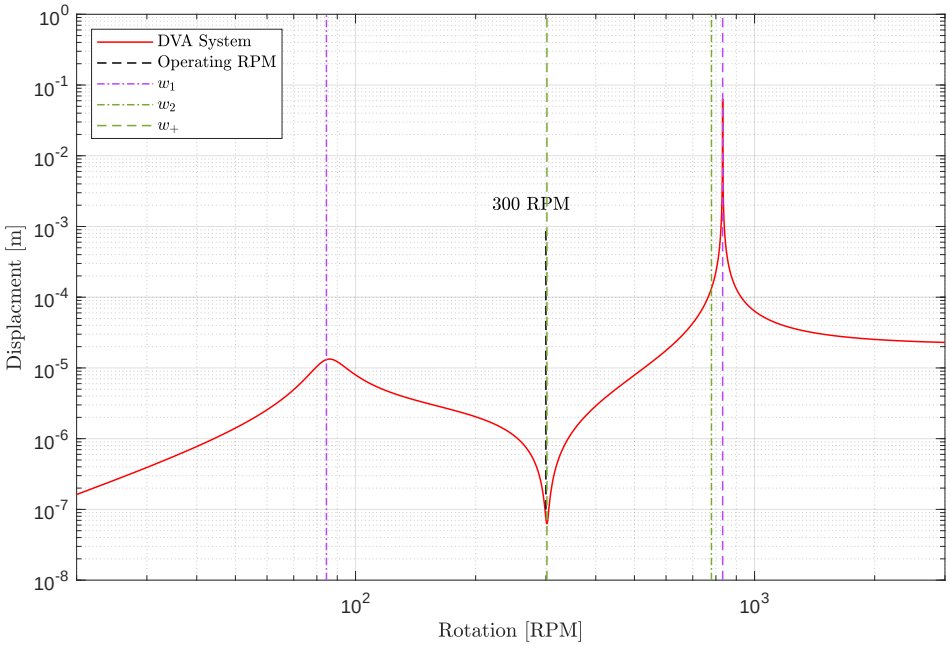

Figure 4: Displacement of the washing machine.

Part 3e

Figure 5 ph

ph

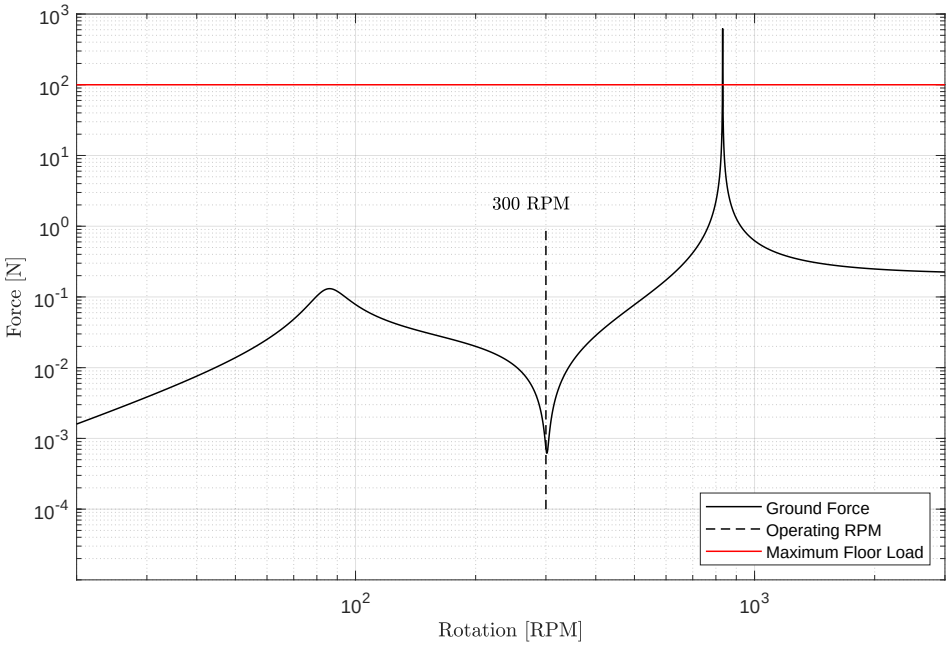

Figure 5: Force applied to the floor.

Problem 4 - Washing Machine Abuse Testing

The Matlab code for this problem is listed in Appendix 4.

Part 3a

1 Appendix - Matlab Code for Problem 1

```
1
2
3
4  %% Synopsis
5
6  % Problem 1 - Washing Machine Mount Design - 1 Degree of Freedom (DOF)
7
8
9
10 %% Notes
11
12 % The mass of the washing machine "does not go away."
13
14
15
16 %% Environment
17
18 close all; clear; clc;
19 % restoredefaultpath;
20
21 addpath( genpath( '../00 Support' ), '-begin' );
22
23 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
24 set( 0, 'DefaultFigureWindowStyle', 'normal' );
25 set( 0, 'DefaultLineLineWidth', 0.8 );
26 set( 0, 'DefaultTextInterpreter', 'Latex' );
27
28 format LongG;
29
30 pause( 1 );
31
32
33
34 %% Anonymous Function Definitions
35
36 h_force = @( force_mass, rotation_speed, load_distance  )  force_mass .* rotation_speed.^2 .*
       load_distance;
37
38
39
40 %% Washing Machine Information
41
42 machine.mass = 100;  % kg
43 machine.rotation_speed = 31.4;  % radians\s or 300 rotations per minute (RPM)
44 machine.load_mass = 10;  % kg
45 machine.static_displacement = 1e-2;  % m
46
47
48
49 %% Load Force Information
50
51 dynamic_force.mass = 1;  % kg
52 dynamic_force.radius = 0.5;  % m
53
54 % Maximum force applied to foundation is 100 N.
55
56
57
58 %% Part 1a
59
60 rpm = 0:1:350;
61     rpm_conversion_to_radians_per_second = 0.10472;
62         angular_velocity = rpm .* rpm_conversion_to_radians_per_second;  % radians\s
63
64
65 % Characteristics of the dynamic load.
66 force_frequency = 300 * rpm_conversion_to_radians_per_second / ( 2 * pi );  % 5 Hz
67     h_force( dynamic_force.mass, 300*rpm_conversion_to_radians_per_second, dynamic_force.radius )
           ;  % 493.5 N
68
69
70 figure( 'Name', 'Problem 1a - Load Force Versus Angular Velocity' ); ...
71     plot( rpm, h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ) );  hold on;
72     plot( 300, 494, 'Marker', '.', 'MarkerSize', 20 );
73     line( [ 300, 300 ], [ 400 600 ], 'Color', 'r' );
```

```
74      text( 305, 494, '493.5 N' );  grid on;
75      xlabel( 'Angular Speed [RPM]' );  ylabel( 'Load Force [N]' );
76      axis( [ -10 355  -5 705 ] );
77
78
79
80  %% Part 1b
81
82  maximum_allowable_displacement = 1e-2;  % m
83      ks = ( machine.load_mass * 9.81) / maximum_allowable_displacement;  % 9,810 N\m
84
85
86  total_mass = machine.mass + machine.load_mass;  % 110 kg
87      wo = sqrt( ks / total_mass );  % 9.4 radians\s
88          fo = wo / (2*pi);  % 1.5 Hz - Natural frequency of the mount.
89
90
91
92  %% Part 1c
93
94  frequency_set = 0.1:0.01:1e3;
95      r = frequency_set ./ fo;
96
97  damping_ratio = logspace( log10(0.001), log10(1), 6 );
98
99  h_transmissibility = @( epsilon, r )  sqrt( ( 1 + (2.*epsilon.*r).^2 ) ./ ( ( 1 - r.^2 ).^2 + (
        2.*epsilon.*r ).^2 ) );
100
101
102 figure( 'Name', 'Problem 1c - Transmission Loss' ); ...
103     hold on;
104     %
105     for index = 1:1:numel( damping_ratio )
106         plot( r, 10*log10( h_transmissibility( damping_ratio( index ), r ) ) );
107     end
108     %
109     xlabel( 'Frequency Ratio [$\frac{f}{fo}$;  unitless]' );  ylabel( 'Tranmission Loss [dB]' );
110         legend( '0.001', '0.004', '0.015', '0.063', '0.25', '1', 'Location', 'NorthEast' );
111     axis( [ 0.1 665  -60 25 ] );
112     grid on;  box on;
113     set( gca, 'XScale', 'log' );
114     %
115     % set( gcf, 'units', 'point', 'pos', [ 200 200    493*0.8 744*0.5 ] );
116     %     pos = get( gcf, 'Position' );
117     %         set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [pos(3)
        , pos(4)] );
118     %             if ( PRINT_FIGURES == 1 )
119     %                 print(gcf, 'Homework_3_Part_1c', '-dpdf', '-r0' );
120     %             end
121
122
123
124 %% Part 1d
125
126 rpm = 0:1:350;  % rotations per minute
127     rpm_conversion_to_radians_per_second = 0.10472;  % radians\s
128         angular_velocity = rpm .* rpm_conversion_to_radians_per_second;  % radians\s
129
130 frequency_set = angular_velocity / (2*pi);
131     r = frequency_set ./ fo;
132
133
134 figure( 'Name', 'Problem 1d - Applied Force to Foundation' ); ...
135     hold on;
136     %
137     for index = 1:1:numel( damping_ratio )
138         plot( rpm, h_transmissibility( damping_ratio( index ), r ) .* h_force( dynamic_force.mass
            , angular_velocity, dynamic_force.radius ) );
139     end
140     %
141     line( [ 300, 300 ], [ 2 1e4 ], 'Color', 'k', 'LineStyle', '--' );
142         text( 305, 2, '5 Hz' );  grid on;
143     line( [ 0 350 ], [ 100 100 ], 'Color', 'k', 'LineStyle', '--' );
144         text( 0, 205, '100 N' );  grid on;
145     xlabel( 'Angular Speed [RPM]' );  ylabel( 'Force Applied to Foundation [N]' );
146         legend( '0.001', '0.004', '0.015', '0.063', '0.25', '1', 'Location', 'South' );
147     axis( [ -10 355  1 2e4 ] );
148     grid on;  box on;
```

```
149     set( gca, 'YScale', 'log' );
150     %
151     % set( gcf, 'units', 'point', 'pos', [ 200 200    493*0.8 744*0.5 ] );
152     %     pos = get( gcf, 'Position' );
153     %         set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [pos(3)
            , pos(4)] );
154     %             if ( PRINT_FIGURES == 1 )
155     %                 print(gcf, 'Homework_3_Part_1d', '-dpdf', '-r0' );
156     %             end
157
158
159 %% Part 1e
160
161 % The best damping ratio is 0.25.
162
163 C = 0.25 * 2*sqrt( ks * total_mass );  % 519.4 kg\s
164
165 % The units of a viscous damping coefficient are Newton-seconds per meter (Ns/m) or kilograms per
        second (kg/s).
166
167
168 m = total_mass;
169 k = [ ks  0 ];
170 dampings = [ C  0 ];
171     admittance = nDOF_direct_solution( m, k, dampings, frequency_set, 'admittance' );
172
173
174 figure( 'Name', 'Problem 1e - Admittance' ); ...
175     semilogy( rpm, abs( admittance ) );  grid on;
176     xlabel( 'Angular Speed [RPM]' );  ylabel( 'Admittance [$\frac{m}{N}$]' );
177     set( gca, 'YScale', 'log' );
178     %
179     line( [ 300, 300 ], [ 7e-6 2e-5 ], 'Color', 'k', 'LineStyle', '--' );
180         text( 305, 1.2e-5, '5 Hz' );  grid on;
181     axis( [ -10 355  6e-6 2.5e-4 ] );
182     %
183     % set( gcf, 'units', 'point', 'pos', [ 200 200    493*0.8 744*0.5 ] );
184     %     pos = get( gcf, 'Position' );
185     %         set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [pos(3)
            , pos(4)] );
186     %             if ( PRINT_FIGURES == 1 )
187     %                 print(gcf, 'Homework_3_Part_1e', '-dpdf', '-r0' );
188     %             end
189
190 % return
191
192 %% Part 1f
193
194 displacement = abs( admittance ) .* h_force( dynamic_force.mass, angular_velocity, dynamic_force.
        radius ).';
195
196
197 figure( 'Name', 'Displacement' ); ...
198     plot( rpm, displacement );  grid on;
199     xlabel( 'Angular Speed [RPM]' );  ylabel( 'Displacement [m]' );
200     line( [ 300, 300 ], [ 1e-3 1e-2 ], 'Color', 'k', 'LineStyle', '--' );
201         text( 305, 1e-3, '5 Hz' );  grid on;
202     set( gca, 'YScale', 'log' );
203     axis( [ -10 355  1e-7 2e-2 ] );
204     %
205     % set( gcf, 'units', 'point', 'pos', [ 200 200    493*0.8 744*0.5 ] );
206     %     pos = get( gcf, 'Position' );
207     %         set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [pos(3)
            , pos(4)] );
208     %             if ( PRINT_FIGURES == 1 )
209     %                 print(gcf, 'Homework_3_Part_1f', '-dpdf', '-r0' );
210     %             end
211
212
213
214 %% Clean-up
215
216 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
217     monitors = get( 0, 'MonitorPositions' );
218         if ( size( monitors, 1 ) == 1 )
219             autoArrangeFigures( 3, 4, 1 );
220         elseif ( 1 < size( monitors, 1 ) )
221
```

```
            autoArrangeFigures( 2, 2, 2 );
        end
end

fprintf( 1, '\n\n\n*** Processing Complete ***\n\n\n' );



%% Reference(s)









%% Displacement Versus Frequency Ratio
%
% Fo = 1;
% wf = 4;  % Force frequency, radians\s
%
% lambda = 1;
%     k = (2*pi)/lambda;
%
% phase_offset = 0;
%
% w = 5;
%     r = wf / w;
%
% time_indices = 0:1e-2:5;
%
% h_x = @( Fo, k, wf, time_indices, phase_offset, r, epsilon ) ( Fo./k.*sin(4*time_indices - phase_offset) ) ./ ( sqrt( (1 - r^2)^2 + (2*r*epsilon)^2 ) );
%
% figure( ); ...
%     epsilon = 0;
%         % plot( time_indices, h_x( Fo, k, wf, time_indices, phase_offset, r, epsilon ) );  hold on;
%     %
%     for wf = 10:-1:1
%         wf
%         epsilon = 0.25;
%         r = wf / w;
%             plot( h_x( Fo, k, wf, time_indices, phase_offset, r, epsilon ) );  hold on;
%         keyboard
%     end
%     %
%     legend( '', '' );
%     xlabel( 'Frequency Ratio [WU]' );  ylabel( 'Normalized Maximum Displacment [WU]' );
```

2 Appendix - Matlab Code for Problem 2

```
1
2
3
4  %% Synopsis
5
6  % Problem 2 - Washing Machine Two-stage Mount Design
7
8  % See Lecture 14, Monday, March 3, 2025
9
10
11
12 %% Note(s)
13
14 % A 2 degree-of-freedom (DOF) system will provide isolation performance 1 DOF system.
15
16 % Allowing the top and bottom masses, the washing machine and the raft,
17 % respectively, to each be attached to ground independently.
18
19 % With the force applied to the washing machine and the displacement of the
20 % washing machine, use FSF( :, 1, 1 ).
21
22 % However, if the force is applied elsewhere, than FSF( :, 2, 2, ) might
23 % need to be used.
24
25
26
27 %% Environment
28
29 close all; clear; clc;
30 % restoredefaultpath;
31
32 addpath( genpath( './00 Support' ), '-begin' );
33
34 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
35 set( 0, 'DefaultFigureWindowStyle', 'normal' );
36 set( 0, 'DefaultLineLineWidth', 0.8 );
37 set( 0, 'DefaultTextInterpreter', 'Latex' );
38
39 format ShortG;
40
41 pause( 1 );
42
43
44
45 %% Define Anonymous Functions
46
47 h_f_to_rpm = @( f )  ( f*2*pi ) / 0.10471;
48 h_w_to_rpm = @( w )  h_f_to_rpm( w*2*pi);
49 h_w_to_f = @( w )  w/(2*pi);
50 h_rpm_to_f = @( rpm )  ( rpm * 0.10471 ) / ( 2 * pi );
51
52 % return
53
54 %% Problem 2a
55
56 % See report.
57
58
59
60 %% Problem 2b - Blocked Frequencies of Resonance
61
62 % The frequency of the forcing function is 5 Hz and the amplitude is 493.5 N.
63 fo = 5;  % Hz
64     wo = 2*pi*fo;  % 31.4 radians\s
65
66
67 m1 = 100 + 10;  % kg - Total mass of the washing machine and the load.
68     k1 = 1e2;  % N\m
69     c1 = 5;
70
71 m2 = 100;  % kg - UPPER LIMIT OF 100 kg
72     k2 = 1e2;  % N\m
73     c2 = 5;
74
75 k3 = 1e3;  % Admittance:  9.31e-6 m\N
```

```
%
% Note(s):
%
%   a.) Strong coupling produces a high w+.


w1 = sqrt( ( k1 + k3 ) / m1 );  %  radians\s;  washing machine
    f1 = w1 / (2*pi);
        rpm1 = f1*(2*pi)/0.10471;

w2 = sqrt( ( k2 + k3 ) / m2 );  %  radians\s;  raft
    f2 = w2 / (2*pi);
        rpm2 = f2*(2*pi)/0.10471;

% Note:  If k3 is set to zero, then these frequencies represent the
% resonance frequencies for each mass on their own.



%% Problem 2b - Coupled Frequencies

mu_4 = ( k3^2 / (m1*m2) );  %  (radians\s)^4
    mu = ( mu_4 )^0.25;

w(1) = 0.5*( ( w1^2 + w2^2 )  +  sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
w(2) = 0.5*( ( w1^2 + w2^2 )  -  sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
    w = sqrt( w );


w_minus = w(2);                 %  0.97 radians\s  (lower than w1, 1, and w2, 1.05 radians\s)
    f_minus = w(2)/(2*pi);  %  0.1545 Hz
        rpm_minus = h_f_to_rpm( f_minus );
%
w_plus = w(1);                    %  1.08 radians\s  (higher than w1, 1, and w2, 1.05 radians\s)
    f_plus = w(1)/(2*pi);     %  0.17124 Hz
        rpm_plus = h_f_to_rpm( f_plus );
%
% Note(s):
%
%   The blocked frequencies, w1 and w2, are always between these two frequencies.


clear w mu_4;



%% Problem 2b - Mode Shapes

phi_plus = [ ...
    1; ...
    -(1/mu^2)*sqrt(m1/m2)*(w_plus^2 - w1^2), ...
    ];

phi_minus = [ ...
    1; ...
    (1/mu^2)*sqrt(m1/m2)*(w1^2 - w_minus^2), ...
    ];


% Note(s):
%
% When m1 = m2 and k1 = k3,
%
%   a.)  At w_minus, m1 and m2 move the same amount and are in-phase.
%   b.)  At w_plus, m1 and m2 move the same amount, but are out-of-phase.

% In general, for a 2 DOF system there will be 2 modes (in-phase and out-of-phase).


% With different initial conditions (no forcing), the behaviour is a weighted sum of the two
    modes.



%% Regions of Interest

% 5 regions of interest:

```

```
% 1.)  Low frequency:  w < w_minus:
%
%   Stiffness controlled;  two masses moving together;  strong coupling (k3 high).


% 2.)  Force at a resonance frequency:  w = w_minus (lowest resonance frequency):
%
%   At lowest resonance frequency;  at w_minus mode, masses moving in same direction (in-phase) with same high amplitude


% 3.) Forcing frequency between w_minus and w2


% 4.)  Forcing frequency w = w2


% 5.)  Forcing frequency w < w < w+
%
%   No a special frequency.


% 6.)  Forcing frequency w = w+
%
% At highest resonance frequency;  at w_plus mode, masses moving in opposite directions (out-of-phase) with same high amplitude


% 7.)  Forcing frequency w+ < w
%
% Mass controlled region;  mass 1 (washing machine) moves more than mass 2 (raft);



%% Problem 2c

masses = [ m1 m2 ];
stiffnesses = [ k1  k3  k2 ];
dampings = [ c1  0  c2 ];

rpm = 0:0.1:500;  % rotations per minute
    rpm_conversion_to_radians_per_second = 0.10472;
        angular_velocity = rpm .* rpm_conversion_to_radians_per_second;  % radians\s
            frequencies = angular_velocity / (2*pi);  % Hz

FRF = nDOF_direct_solution( masses, stiffnesses, dampings, frequencies, 'admittance' );


admittance = zeros( numel( frequencies ), 1 );

for index = 1:1:numel( frequencies )
    temp = diag( squeeze( FRF( index, 1, 1 ) ) );  % Check indexing to ensure force on washing machine.
        admittance( index ) = abs( temp );
end
%
clear temp temp2;

washing_machine.admittance = admittance;


admittance = zeros( numel( frequencies ), 1 );

for index = 1:1:numel( frequencies )
    temp = diag( squeeze( FRF( index, 1, 2 ) ) );  % Check indexing to ensure force on washing machine.
        admittance( index ) = abs( temp );
end
%
clear temp temp2;

raft.admittance = admittance;


figure( 'Name', 'Admittance - Magnitude' ); ...
    h1 = loglog( rpm, washing_machine.admittance, 'Color', 'r' );  hold on;
    h2 = loglog( rpm, raft.admittance, 'Color', 'b' );
    h3 = line( [ 300 300 ], [ 1e-6 1e-4 ], 'Color', 'k', 'LineStyle', '--' );  grid on;
```

```
227         text( 220, 1e-4, '5 Hz' );
228     %
229     h4 = line( [ rpm_minus rpm_minus], [ 1e-8 1e1 ], 'Color', [ 1.00, 0.41, 0.16 ], 'LineStyle',
            '-.' );
230     %
231     h5 = line( [ rpm1 rpm1 ], [ 1e-8 1e1 ], 'Color', 'm', 'LineStyle', '-.' );
232     h6 = line( [ rpm2 rpm2 ], [ 1e-8 1e1 ], 'Color', 'm', 'LineStyle', '-' );
233     %
234         h7 = line( [ rpm_plus rpm_plus ], [ 1e-8 1e1 ], 'Color', [ 1.00, 0.41, 0.16 ], 'LineStyle
                ', '-' );
235     %
236     legend( [ h1 h2 h3 h4 h5 h6 h7 ], 'Washing Machine', 'Raft', 'Load Frequency', 'w-', 'w1', '
            w2', 'w+', 'Location', 'NorthEast' );
237     %
238     xlabel( 'Rotation [RPM]' );  ylabel( 'Admittance (Magnitude) [$\frac{m}{N}$]' );
239     axis( [ 6 400  1e-8 1e1 ] );
240     %
241     % set( gcf, 'units', 'point', 'pos', [ 200 200    493*0.8 744*0.5 ] );
242     %     pos = get( gcf, 'Position' );
243     %         set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [pos(3)
            , pos(4)] );
244     %             if ( PRINT_FIGURES == 1 )
245     %                 print(gcf, 'Homework_3_Part_2c', '-dpdf', '-r0' );
246     %             end
247
248
249
250 %% Problem 2d
251
252 dynamic_force.mass = 1;  % kg
253 dynamic_force.radius = 0.5;  % m
254
255 h_force = @( force_mass, rotation_speed, load_distance  )  force_mass .* rotation_speed.^2 .*
        load_distance;
256
257 temp = h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ).';
258
259
260 figure( 'Name', '' ); ...
261     h1 = loglog( rpm, washing_machine.admittance.*temp, 'Color', 'r' );  hold on;
262     h2 = loglog( rpm, raft.admittance.*temp, 'Color', 'b' );
263     h3 = line( [ 300 300 ], [ 1e-6 1e-1 ], 'Color', 'k', 'LineStyle', '--' );  grid on;
264         text( 220, 1e-1, '5 Hz' );
265         legend( [ h1 h2 h3 ], 'Washing Machine', 'Raft', 'Load Frequency', 'Location', 'South' );
266     xlabel( 'Rotation [RPM]' );  ylabel( 'Displacment [m]' );
267     axis( [ 6 400  1e-8 1e1 ] );
268     %
269     % set( gcf, 'units', 'point', 'pos', [ 200 200    493*0.8 744*0.5 ] );
270     %     pos = get( gcf, 'Position' );
271     %         set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [pos(3)
            , pos(4)] );
272     %             if ( PRINT_FIGURES == 1 )
273     %                 print(gcf, 'Homework_3_Part_2d', '-dpdf', '-r0' );
274     %             end
275
276
277
278 %% Problem 2e
279
280 % Displacement of the raft multiplied by the K2 spring constant.
281
282 dynamic_force.mass = 1;  % kg
283 dynamic_force.radius = 0.5;  % m
284
285 h_force = @( force_mass, rotation_speed, load_distance  )  force_mass .* rotation_speed.^2 .*
        load_distance;
286
287 temp = h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ).';
288
289
290 figure( 'Name', 'Force Applied to Ground by Raft' ); ...
291     h1 = loglog( rpm, raft.admittance.*temp*k2, 'Color', 'b' );  hold on;
292     h2 = line( [ 300 300 ], [ 1e-4 1e-1 ], 'Color', 'k', 'LineStyle', '--' );  grid on;
293     h3 = line( [ 6 400 ], [ 100 100 ], 'Color', 'r' );
294         text( 220, 1e-1, '5 Hz' );
295         legend( [ h1 h2 h3 ], 'Raft', 'Load Frequency', 'Maximum Floor Load', 'Location', 'South'
                );
296     xlabel( 'Rotation [RPM]' );  ylabel( 'Force Applied to Ground [N]' );
```

```
297      axis( [ 6 400  1e-5 1e3 ] );
298      %
299      % set( gcf, 'units', 'point', 'pos', [ 200 200    493*0.8 744*0.5 ] );
300      %     pos = get( gcf, 'Position' );
301      %         set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [pos(3)
         , pos(4)] );
302      %             if ( PRINT_FIGURES == 1 )
303      %                 print(gcf, 'Homework_3_Part_2e', '-dpdf', '-r0' );
304      %             end
305
306
307
308  %% Problem 2f
309
310  % return
311
312  %% Problem 2g
313
314  % return
315
316  %% 2 DOF Example
317
318  % masses = [ 50  1 ];
319  % stiffnesses = [ 100e3  1800  0 ];
320  %
321  % dampings = [ 0  0  0 ];
322  %% dampings = [ 0  (1 + 0.1*1j)  (1 + 0.1*1j) ];
323  %
324  % frequencies = 0:0.01:40;
325  %
326  % FRF = nDOF_direct_solution( masses, stiffnesses, dampings, frequencies, 'admittance' );  %
       4001-by-2-by-2
327  %
328  %
329  % admittance = zeros( numel( frequencies ), 1 );
330  %
331  % for index = 1:1:numel( frequencies )
332  %     temp = diag( squeeze( FRF( index, :, : ) ) );
333  %         temp2 = temp(1) + temp(2)*1j;
334  %         admittance( index ) = abs( temp2 );
335  % end
336  %
337  % clear temp1 temp2;
338  %
339  %
340  %% figure( ); ...
341  %%     semilogy( frequencies, admittance );  grid on;
342  %%     xlabel( 'Frequency [Hz]' );  ylabel( 'Admittance [$\frac{m}{N}$]' );
343  %
344  %
345  % figure( 'Name', 'Admittance' ); ...
346  %     semilogy( frequencies, abs( FRF( :, 1, 1 ) ) );  hold on;
347  %     semilogy( frequencies, abs( FRF( :, 1, 2 ) ) );
348  %     semilogy( frequencies, abs( FRF( :, 2, 1 ) ) );
349  %     semilogy( frequencies, abs( FRF( :, 2, 2 ) ) );  grid on;
350  %         legend( 'M1 Movement with F1', 'M1 Movement with F2', 'M2 Movement with F1', 'M2
       Movement with F2', 'Location', 'North' );
351  %     xlabel( 'Frequency [Hz]' );  ylabel( 'Admittance [$\frac{m}{N}$]' );
352  %     axis( [ 5.5 10.5  1e-7 1e2 ] );
353
354
355  % figure( 'Name', 'Dynamic Stiffness (Reciprocal of Admittance)' ); ...
356  %     semilogy( frequencies, 1 ./ abs( FRF( :, 1, 1 ) ) );  hold on;
357  %     semilogy( frequencies, 1 ./ abs( FRF( :, 1, 2 ) ) );
358  %     semilogy( frequencies, 1 ./ abs( FRF( :, 2, 1 ) ) );
359  %     semilogy( frequencies, 1 ./ abs( FRF( :, 2, 2 ) ) );  grid on;
360  %         legend( 'M1 Movement with F1', 'M1 Movement with F2', 'M2 Movement with F1', 'M2
       Movement with F2', 'Location', 'North' );
361  %     xlabel( 'Frequency [Hz]' );  ylabel( 'Admittance [$\frac{m}{N}$]' );
362
363
364
365  %% Clean-up
366
367  if ( ~isempty( findobj( 'Type', 'figure' ) ) )
368      monitors = get( 0, 'MonitorPositions' );
369          if ( size( monitors, 1 ) == 1 )
370              autoArrangeFigures( 3, 4, 1 );
```

```
        elseif ( 1 < size( monitors, 1 ) )
            autoArrangeFigures( 2, 2, 2 );
        end
end

fprintf( 1, '\n\n\n*** Processing Complete ***\n\n\n' );



%% Reference(s)

% https://tex.stackexchange.com/questions/295059/three-tables-side-by-side-on-one-page



%% Reference Code

% admittance = zeros( numel( frequencies ), 1 );
%
% for index = 1:1:numel( frequencies )
%     temp = diag( squeeze( FRF( index, 1, 1 ) ) );  % Check indexing to ensure force on washing machine.
%         admittance( index ) = unwrap( angle ( temp ) ) * 180 / pi;
% end
% %
% clear temp temp2;
%
% washing_machine.admittance_phase = admittance;
%
%
% admittance = zeros( numel( frequencies ), 1 );
%
% for index = 1:1:numel( frequencies )
%     temp = diag( squeeze( FRF( index, 1, 2 ) ) );  % Check indexing to ensure force on washing machine.
%         admittance( index ) = unwrap( angle ( temp ) ) * 180 / pi;
% end
% %
% clear temp temp2;
%
% raft.admittance_phase = admittance;
%
% figure( 'Name', 'Admittance - Phase' ); ...
%     h1 = semilogx( rpm, washing_machine.admittance_phase, 'Color', 'r' );  hold on;
%     h2 = semilogx( rpm, raft.admittance_phase   , 'Color', 'b', 'LineStyle', '--' );  grid on;
%     h3 = line( [ 300 300 ], [ -200 200 ], 'Color', 'k', 'LineStyle', '--' );  grid on;
%     %
%     h4 = line( [ rpm_minus rpm_minus], [ -200 200 ], 'Color', 'g', 'LineStyle', '-.' );
%     %
%     line( [ rpm1 rpm1 ], [ -200 200 ], 'Color', 'm', 'LineStyle', '--' );
%     line( [ rpm2 rpm2 ], [ -200 200 ], 'Color', 'm', 'LineStyle', '--' );
%     %
%     h5 = line( [ rpm_plus rpm_plus ], [ -200 200 ], 'Color', 'g', 'LineStyle', '-' );
%     %
%     legend( [ h1 h2 h3 h4 h5 ], 'Washing Machine', 'Raft', 'Load Frequency', 'w-', 'w+', 'Location', 'NorthEast' );
%     %
%     xlabel( 'Rotation [RPM]' );  ylabel( 'Admittance [$\frac{m}{N}$]' );
%     xlim( [ 6 400 ] );
%     % axis( [ 6 400  1e-8 1e1 ] );
%     %
%     % set( gcf, 'units', 'point', 'pos', [ 200 200    493*0.8 744*0.5 ] );
%     %     pos = get( gcf, 'Position' );
%     %         set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [pos(3), pos(4)] );
%     %             if ( PRINT_FIGURES == 1 )
%     %                 print(gcf, 'Homework_3_Part_2c', '-dpdf', '-r0' );
%     %             end
```

3 Appendix - Matlab Code for Problem 3

```
1
2
3  % To do:
4
5  % remove comment statements
6
7  % verify indices for required values
8
9
10
11 %% Synopsis
12
13 % Problem 3 - Washing Machine Dynamic Vibration Absorber (DVA) Design
14
15
16
17 %% Note(s)
18
19 % At 300 RPM, the load frequency, this system will isolated vibration more
20 % than the 2 DOF and 1 DOF systems.
21
22 % The DVA behaves like a Helmholtz resonator, working as a mechnical notch
23 % filter at a particular frequency.
24
25 % DVAs work well in systems that have one dominate mode.  A good
26 % example of this is building sway and its compensation.  Based on the
27 % design and construction of a building, it will typically have a single,
28 % dominate mode.
29
30 % DVA is useful if you can not get away from the resonance frequency;  can
31 % not tune a 2 DOF system when the resonance frequency is away from the
32 % forcing frequency.
33
34 % DVA introduces two resonances for the original resonance.  These new
35 % resonances will produce greater motion at the new resonance frequencies.
36
37
38
39 %% Environment
40
41 close all; clear; clc;
42 % restoredefaultpath;
43
44 addpath( genpath( './00 Support' ), '-begin' );
45
46 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
47 set( 0, 'DefaultFigureWindowStyle', 'normal' );
48 set( 0, 'DefaultLineLineWidth', 0.8 );
49 set( 0, 'DefaultTextInterpreter', 'Latex' );
50
51 format ShortG;
52
53 pause( 1 );
54
55
56
57 %% Define Anonymous Functions
58
59 h_rpm_to_f = @( rpm )  ( rpm * 0.10471 ) / ( 2 * pi );
60 h_rpm_to_w = @( rpm )  ( rpm * 0.10471 );
61
62 h_f_to_rpm = @( f )  ( f*2*pi ) / 0.10471;
63
64 h_w_to_f = @( w )  w/(2*pi);
65 h_w_to_rpm = @( w )  h_f_to_rpm( w*2*pi);
66
67
68 h_force = @( force_mass, rotation_speed, load_distance  )  force_mass .* rotation_speed.^2 .*
       load_distance;
69
70
71
72 %% Washing Machine Information
73
74 machine.mass = 100;  % kg
```

```
machine.rotation_speed = 31.4;  % radians\s or 300 rotations per minute (RPM)
machine.load_mass = 10;  % kg
machine.static_displacement = 1e-2;  % m



%% Load Force Information

dynamic_force.mass = 1;  % kg
dynamic_force.radius = 0.5;  % m

% Maximum force applied to foundation is 100 N.



%% Dynamic Vibration Absorber (DVA)

maximum_allowable_displacement = 1e-2;  % m
    ks = ( machine.load_mass * 9.81) / maximum_allowable_displacement;  % 9,810 N\m


m = 110;  % kg;  washing machine mass and load mass
k = ks;  % N\m

wo = sqrt( k / m);  % 9.4 radians\s
    fo = wo/(2*pi);  % 1.5 Hz


m_dva = 15;  % kg
k_dva = 1e5;  % N\m

wo_dva = sqrt( k_dva / m_dva );  % 81.7 radians\s
    fo_dva = wo_dva/(2*pi);  % 13.0 Hz



%% Blocked Frequencies (Equation 9.1 of the Notes)

m1 = m_dva;
k1 = 0;  % Spring above DVA mass;  does not exist;  set to zero.

m2 = m;
k2 = k;

k3 = k_dva;


w1 = sqrt( ( k1 + k3 ) / m1 );  % Smaller than the system resonant frequency.
    f1 = w1/(2*pi);

w2 = sqrt( ( k2 + k3 ) / m2 );  %  Greater than the system resonant frequency.
    f2 = w2/(2*pi);



%% Coupled Frequencies (Equation 9.15 of the Notes)

mu_4 = ( k3^2 / (m1*m2) );
    mu = ( mu_4 )^0.25;

w(1) = 0.5*( ( w1^2 + w2^2 )  +  sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
w(2) = 0.5*( ( w1^2 + w2^2 )  -  sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
    w = sqrt( w );


w_minus = w(2);                % 8.2 radians\s  (lower than 42.4 and 45.1 radians\s)
    f_minus = w(2)/(2*pi);
%
w_plus = w(1);                   % 12.6 radians\s (higher than 42.4 and 45.1 radians\s)
    f_plus = w(1)/(2*pi);


% The blocked frequencies are always between these two frequencies.

% return

%% Mode Shapes (Equations 9.18 and 9.19 of the Notes)

```

```
phi_plus = [ ...
    1; ...
    -(1/mu^2)*sqrt(m1/m2)*(w_plus^2 - w1^2), ...
    ];

phi_minus = [ ...
    1; ...
    (1/mu^2)*sqrt(m1/m2)*(w1^2 - w_minus^2), ...
    ];



%% Plot (1) Admittance Using Equations from DVA Lecture

MAXIMUM_RPM = 3e3;

F0 = 1;

w = 0:0.01:h_rpm_to_w( MAXIMUM_RPM );
    f = w/(2*pi);
        rpm = h_f_to_rpm( f );


m = [ 15 110 ];
k = [  0  k_dva.*(1+0.1*1i)  k2.*(1+0.2*1i)  ];


admittance_modified = F0 ./ ( m(1)*m(2) )*k(2) ./ (( w.^2 - w_plus^2 ) .* ( w.^2 - w_minus^2) );
admittance_original = F0 ./ m(2) * 1 ./ ( w2^2 - w.^2 );


figure( 'Name', 'Admittance - Equations from DVA Lecture' ); ...
    h1 = loglog( rpm, abs( admittance_original ) );  hold on;
    h2 = loglog( rpm, abs( admittance_modified ) );
    h3 = line( [ 300 300 ], [ 1e-6 1e-2 ], 'Color', 'k', 'LineStyle', '--' );
        text( 220, 2e-2, '300 RPM' );
    h4 = line( [ h_f_to_rpm( f_minus )  h_f_to_rpm( f_minus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
        0.27, 1.00 ], 'LineStyle', '-.' );

    h5 = line( [ h_f_to_rpm( f1 )  h_f_to_rpm( f1 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
        ], 'LineStyle', '-.' );
    h6 = line( [ h_f_to_rpm( f2 )  h_f_to_rpm( f2 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
        ], 'LineStyle', '--' );

    h7 = line( [ h_f_to_rpm( f_plus )  h_f_to_rpm( f_plus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
        0.27, 1.00 ], 'LineStyle', '--' );    grid on;

    axis( [ 5 3e3  1e-7 1 ] );
        legend( [ h1 h2 h3 h4 h5 h6 h7 ], 'Original System', 'DVA System', 'Operating RPM', ...
            '$w_-$', '$w_1$', ...
            '$w_2$', '$w_+$', ...
            'Location', 'NorthWest', 'Interpreter', 'Latex' );
    xlabel( 'Rotation [RPM]' );  ylabel( 'Admittance [$\frac{m}{N}$]' );



%% Plot (2) Admittance Using nDOF Function

% Damping can be added by using a dampings vector or appending a complex
% value to the respective spring stiffness.  THESE ARE NOT INTERCHANGABLE.


rpm = 0:0.1:MAXIMUM_RPM;
    f = h_rpm_to_f( rpm );
        w = f ./ (2*pi);

FRF = nDOF_direct_solution( m, k, [ 0 0 0 ], f, 'admittance' );  % Symmetric matrix.
%
squeeze( FRF( numel( f ), :, :) );
%
% (1, 1) - Force on M1 and its associated displacement.
% (2, 2) - Force on M2 and its associated displacement.
%
% (1, 2) - Force on M1 and the displacement of M2.
% (2, 1) - Force on M2 and the displacment on M1.
%
%   These are interchangeable;  the same result.
```

```
227
228 figure( 'Name', 'Admittance - nDOF Calculated' ); ...
229     h1 = loglog( h_f_to_rpm( f ), abs( FRF( :, 1, 1 ) ), 'Color', 'r' );  hold on;
230     h2 = line( [ 300 300 ], [ 1e-6 1e-2 ], 'Color', 'k', 'LineStyle', '--' );
231         text( 220, 2e-2, '300 RPM' );
232     h3 = line( [ h_f_to_rpm( f_minus )  h_f_to_rpm( f_minus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
        0.27, 1.00 ], 'LineStyle', '-.' );
233
234     h4 = line( [ h_f_to_rpm( f1 )  h_f_to_rpm( f1 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
        ], 'LineStyle', '-.' );
235     h5 = line( [ h_f_to_rpm( f2 )  h_f_to_rpm( f2 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
        ], 'LineStyle', '--' );
236
237     h6 = line( [ h_f_to_rpm( f_plus )  h_f_to_rpm( f_plus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
        0.27, 1.00 ], 'LineStyle', '--' );    grid on;
238
239     axis( [ 2e1 MAXIMUM_RPM  1e-7 1 ] );
240         legend( [ h1 h2 h3 h4 h5 h6 ], 'DVA System', 'Operating RPM', ...
241             '$w_-$', '$w_1$', ...
242             '$w_2$', '$w_+$', ...
243             'Location', 'NorthWest', 'Interpreter', 'Latex' );
244     xlabel( 'Rotation [RPM]' );  ylabel( 'Admittance [$\frac{m}{N}$]' );
245
246
247 temp2 = abs( FRF( :, 1, 1 ) );
248     temp2( 3001 )
249
250
251 %% Displacement
252
253 dynamic_force.mass = 1;  % kg
254 dynamic_force.radius = 0.5;  % m
255
256 % angular_velocity = f / (2*pi);
257 % rpm = h_f_to_rpm( f );
258
259 temp = h_force( dynamic_force.mass, w, dynamic_force.radius ).';
260
261
262 figure( 'Name', 'Displacement' ); ...
263     h1 = loglog( rpm, abs( FRF( :, 1, 1 ) ).*temp, 'Color', 'r' );  hold on;
264     h2 = line( [ 300 300 ], [ 1e-7 1e-3 ], 'Color', 'k', 'LineStyle', '--' );  grid on;
265         text( 220, 2e-3, '300 RPM' );
266     h4 = line( [ h_f_to_rpm( f_minus )  h_f_to_rpm( f_minus ) ], [ 1e-8 1e3 ], 'Color', [ 0.72,
        0.27, 1.00 ], 'LineStyle', '-.' );
267
268     h5 = line( [ h_f_to_rpm( f1 )  h_f_to_rpm( f1 ) ], [ 1e-8 1e3 ], 'Color', [ 0.47, 0.67, 0.19
        ], 'LineStyle', '-.' );
269     h6 = line( [ h_f_to_rpm( f2 )  h_f_to_rpm( f2 ) ], [ 1e-8 1e3 ], 'Color', [ 0.47, 0.67, 0.19
        ], 'LineStyle', '--' );
270
271     h7 = line( [ h_f_to_rpm( f_plus )  h_f_to_rpm( f_plus ) ], [ 1e-8 1e3 ], 'Color', [ 0.72,
        0.27, 1.00 ], 'LineStyle', '--' );    grid on;
272
273         legend( [ h1 h2 h3 h4 h5 h6 ], 'DVA System', 'Operating RPM', ...
274             '$w_-$', '$w_1$', ...
275             '$w_2$', '$w_+$', ...
276             'Location', 'NorthWest', 'Interpreter', 'Latex' );
277
278     xlabel( 'Rotation [RPM]' );  ylabel( 'Displacment [m]' );
279     axis( [ 2e1 MAXIMUM_RPM  1e-8 1 ] );
280
281
282 temp2 = abs( FRF( :, 1, 1 ) ).*temp;
283     temp2( 3001 )
284
285 % return
286
287 %% Ground Force
288
289 dynamic_force.mass = 1;  % kg
290 dynamic_force.radius = 0.5;  % m
291
292
293 figure( 'Name', 'Ground Force' ); ...
294     h1 = loglog( rpm, abs( FRF( :, 1, 1 ) ).*h_force( dynamic_force.mass, w, dynamic_force.radius
        ).'.*k2, 'Color', 'k' );  hold on;
295
```

```
296     h2 = line( [ 300 300 ], [ 1e-4 1e-0 ], 'Color', 'k', 'LineStyle', '--' );  grid on;
297         text( 220, 2e-0, '300 RPM' );
298     h3 = line( [ 6 MAXIMUM_RPM ], [ 100 100 ], 'Color', 'r' );
299         legend( [ h1 h2 h3 ], 'Ground Force', 'Operating RPM', 'Maximum Floor Load', 'Location',
                'SouthEast' );
300     xlabel( 'Rotation [RPM]' );  ylabel( ' Force [N]' );
301     axis( [ 2e1 MAXIMUM_RPM  1e-5 1e3 ] );
302
303
304 temp2 = abs( FRF( :, 1, 1 ) ).*temp*k2;
305     temp2( 3001 )
306
307
308
309 %% Clean-up
310
311 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
312     monitors = get( 0, 'MonitorPositions' );
313         if ( size( monitors, 1 ) == 1 )
314             autoArrangeFigures( 3, 4, 1 );
315         elseif ( 1 < size( monitors, 1 ) )
316             autoArrangeFigures( 2, 2, 2 );
317         end
318 end
319
320 fprintf( 1, '\n\n\n*** Processing Complete ***\n\n\n' );
321
322
323
324 %% Reference(s)
```

4 Appendix - Matlab Code for Problem 4

```
1
2
3
4  %% Synopsis
5
6  % Problem 4 - Washing Machine Abuse Testing
7
8
9
10 %% Environment
11
12 close all; clear; clc;
13 % restoredefaultpath;
14
15 addpath( genpath( './00 Support' ), '-begin' );
16
17 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
18 set( 0, 'DefaultFigureWindowStyle', 'normal' );
19 set( 0, 'DefaultLineLineWidth', 0.8 );
20 set( 0, 'DefaultTextInterpreter', 'Latex' );
21
22 format ShortG;
23
24 pause( 1 );
25
26
27
28 %% Impulse Signal
29
30 time_step = 1e-4;  % s
31 net_time = 10;  % s
32     time_indices = 0:time_step:( net_time - time_step );
33
34 impulse = zeros( size( time_indices ) );
35     impulse( 4/time_step ) = 1./time_step;
36         brick_impulse = 5 .* impulse;
37         % brick_impulse = 50 .* impulse;
38
39
40 % figure( 'Name', 'Unit Impulse Signal' ); ...  % Figure 1
41 %     plot( time_indices, impulse );  grid on;
42 %         legend( 'Unit Impulse Signal', 'Location', 'NorthEast' );
43 %     xlabel( 'Time [s]' );  ylabel( 'Force [N]' );
44 %     axis( [ 0 net_time  0 125 ] );
45
46 % return
47
48 %% 1 DOF System Impulse Function
49
50 m = 5;  % kg
51
52 % Initial conditions of 1 DOF system.
53 xo = 0;  % m
54 vo = 1 / m;  % m\s
55
56 wd = 31.4;  % radians\s or 5 Hz or 300 RPM
57
58
59 ks = 9810;  % N\m
60     wo = sqrt( ks / 110 );  % 9.4 radians\s
61         fo = wo / (2*pi);  % 1.5 Hz
62
63
64 epsilon = 0.25;
65     C = epsilon * 2*sqrt( ks * 110 );  % 519.4 kg\s
66
67
68 h_unit_impulse = @( washing_machine_mass, wd, wo, epsilon, t )  ( 1./( washing_machine_mass.*wd)
       )  .*  exp( -wo.*epsilon.*t ) .* sin( wd.*t );  % mvo = 1 kg\(m s)
69
70
71
72 %% 1 DOF System Impulse Response
73
74 impulse_response = h_unit_impulse( 100, wd, wo, epsilon, time_indices );
```

```
%
figure( 'Name', '1 DOF System Impulse Response' ); ... % Figure 2
    plot( time_indices, impulse_response );  grid on;
    xlabel( 'Time [s]' );  ylabel( 'Admittance [$\frac{m}{N}$]' );
    axis( [ -0.1 3  -2.5e-4 3.0e-4 ] );


response_to_brick_perturbation = conv( brick_impulse, impulse_response ) * time_step;
    time_indices_perturbation = ( 0:1:( numel( response_to_brick_perturbation ) - 1 ) ) .*
        time_step;
%
figure( 'Name', '1 DOF Impulse Response' ); ... % Figure 3
    plot( time_indices_perturbation, response_to_brick_perturbation );  grid on;
    xlabel( 'Time [s]' );  ylabel( 'Displacement [m]' );
    axis( [ -0.1 5  -1.5e-3 1.5e-3 ] );



%% Sinusoidal Impulse Response

% sinusoid_input = 493.5 * sin( 2 * pi * 5 * time_indices );
sinusoid_input = 1 * sin( 2 * pi * 5 * time_indices );
% sinusoid_input = 493.5 * sin( 2 * pi * 1 * time_indices );
    sinusoid_input( 1:1:( 1 / time_step ) ) = 0;
    sinusoid_input( ( 8 / time_step ):1:( 10 / time_step ) ) = 0;
%
figure( 'Name', 'Sinusoidal Input' ); ... % Figure 4
    plot( time_indices, sinusoid_input );  grid on;
    xlabel( 'Time [s]' );  ylabel( 'Force [N]' );
    % xlim( [ 0 2.5 ] );
    ylim( [ -600 600 ] );


response_to_sinusoidal_forcing = conv( sinusoid_input, impulse_response ) * time_step;
    time_indices_sinusoidal_forcing = ( 0:1:( numel( response_to_sinusoidal_forcing ) - 1 ) ) .*
        time_step;
%
figure( 'Name', '1 DOF Sinusoidal Forcing Response' ); ... % Figure 5
    plot( time_indices_sinusoidal_forcing, response_to_sinusoidal_forcing );  grid on;
    xlabel( 'Time [s]' );  ylabel( 'Displacement [m]' );
    % xlim( [ 0 2.5 ] );



%% Combined Response

joint_signal = sinusoid_input + brick_impulse;
%
figure( 'Name', 'Combined Signal' ); ... % Figure 6
    plot( time_indices, joint_signal );  hold on;
    plot( time_indices, sinusoid_input, 'LineStyle', '--' );
    plot( time_indices, impulse, 'LineStyle', '--' );
    xlabel( 'Time [s]' );  ylabel( 'Force [N]' );
    % xlim( [ 0 2.5 ] );
    % ylim( [ -600 600 ] );


response_to_combined_forcing = conv( impulse_response, joint_signal ) .* time_step;
    time_indices_sinusoidal_forcing = ( 0:1:( numel( response_to_brick_perturbation ) - 1 ) ) .*
        time_step;
%
figure( 'Name', '1 DOF Combined Forcing Response' ); ... % Figure 7
    plot( time_indices_sinusoidal_forcing, response_to_combined_forcing );  grid on;
    xlabel( 'Time [s]' );  ylabel( 'Displacement [m]' );
    % xlim( [ 0 2.5 ] );



%% Plot Difference

figure( 'Name', '1 DOF Combined Forcing Response' ); ... % Figure 7
    plot( time_indices_sinusoidal_forcing, response_to_combined_forcing -
        response_to_sinusoidal_forcing );  grid on;
    xlabel( 'Time [s]' );  ylabel( 'Displacement [m]' );
    % xlim( [ 0 2.5 ] );



```

```
149 %% Clean-up
150
151 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
152     monitors = get( 0, 'MonitorPositions' );
153         if ( size( monitors, 1 ) == 1 )
154             autoArrangeFigures( 3, 4, 1 );
155         elseif ( 1 < size( monitors, 1 ) )
156             autoArrangeFigures( 2, 2, 2 );
157         end
158 end
159
160 fprintf( 1, '\n\n\n*** Processing Complete ***\n\n\n' );
161
162
163
164 %% Reference(s)
```